

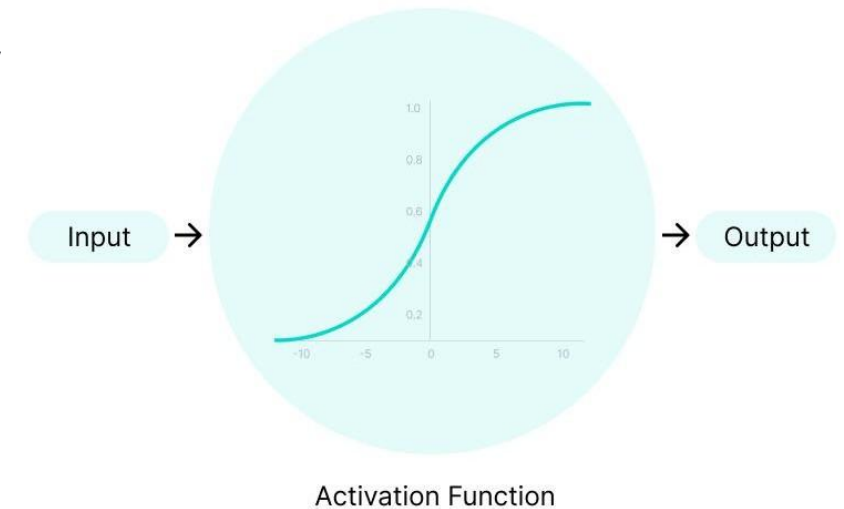


Activation Functions, Loss Functions

CS-4025: Deep Learning

Why do Neural Networks Need an Activation Function?

- The purpose of an activation function is to add non-linearity to the neural network.
- Activation functions introduce an additional step at each layer during the forward propagation,
- Let's suppose we have a neural network working *without* the activation functions. In that case, every neuron will only be performing a linear transformation on the inputs using the weights and biases.
- It's because it doesn't matter how many hidden layers we attach in the neural network; all layers will behave in the same way because the composition of two linear functions is a linear function itself.
- Although the neural network becomes simpler, learning any complex task is impossible, and our model would be just a linear regression model.



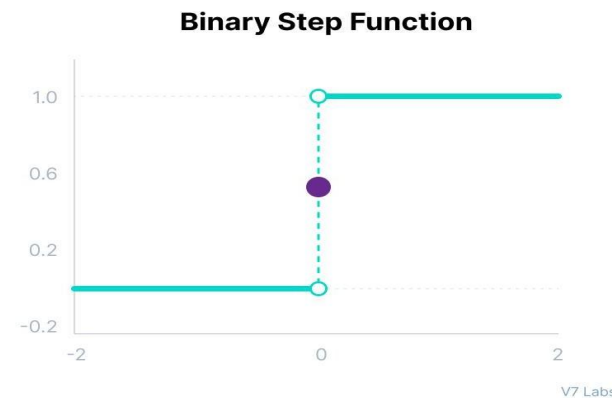
Types of Neural Networks Activation Functions

- Binary Step Function

- Binary step function depends on a threshold value that decides whether a neuron should be activated or not.
- The input fed to the activation function is compared to a certain threshold; if the input is greater than it, then the neuron is activated, else it is deactivated, meaning that its output is not passed on to the next hidden layer.
- Here are some of the limitations of binary step function:
 - It cannot provide multi-value outputs—for example, it cannot be used for multi-class classification problems.
 - The gradient of the step function is zero, which causes a hindrance in the backpropagation process.

Binary step

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$



Linear Activation Function

The linear activation function is also known as *Identity Function* where the activation is proportional to the input.

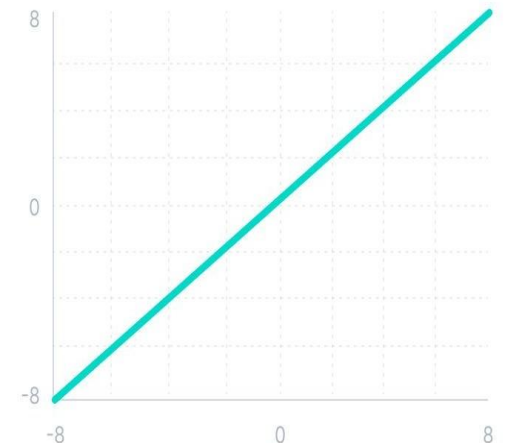
However, a linear activation function has two major problems :

- It's not possible to use backpropagation as the derivative of the function is a constant and has no relation to the input x .
- All layers of the neural network will collapse into one if a linear activation function is used. No matter the number of layers in the neural network, the last layer will still be a linear function of the first layer. So, essentially, a linear activation function turns the neural network into just one layer.

Linear

$$f(x) = x$$

Linear Activation Function



Non-Linear Activation Functions

- The linear activation function shown above is simply a linear regression model.
- Because of its limited power, this does not allow the model to create complex mappings between the network's inputs and outputs.
- Non-linear activation functions solve the following limitations of linear activation functions:
 - They allow backpropagation because now the derivative function would be related to the input, and it's possible to go back and understand which weights in the input neurons can provide a better prediction.
 - They allow the stacking of multiple layers of neurons as the output would now be a non-linear combination of input passed through multiple layers. Any output can be represented as a functional computation in a neural network.

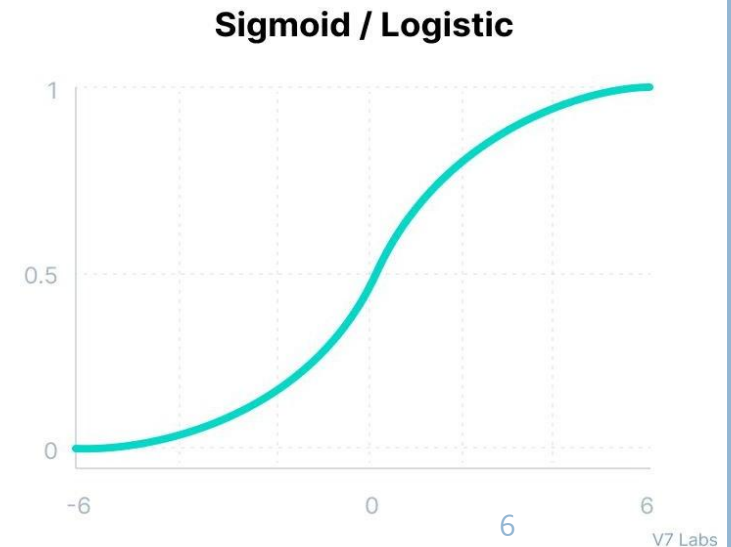
Non-Linear Neural Networks Activation Functions

Sigmoid / Logistic Activation Function

- This function takes any real value as input and outputs values in the range of 0 to 1.
- The larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to 0.0, as shown below.

Sigmoid / Logistic

$$f(x) = \frac{1}{1 + e^{-x}}$$



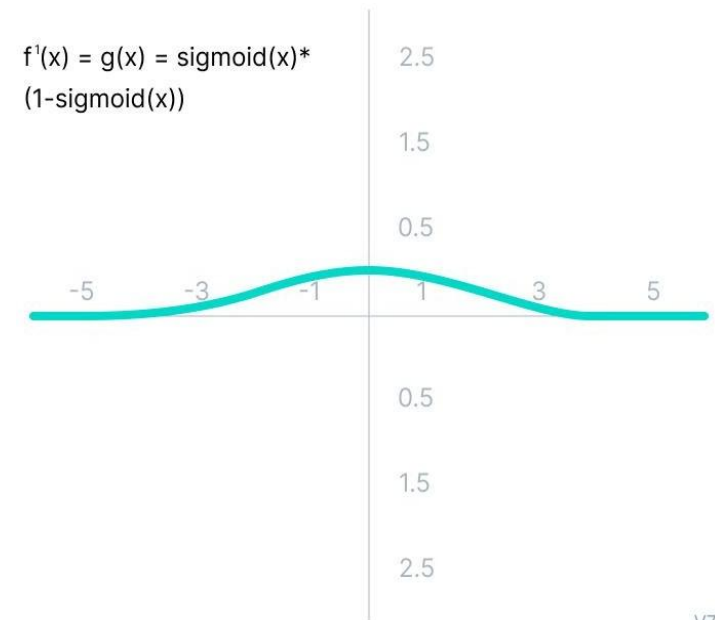
Why sigmoid?

- It is commonly used for models where we have to predict the probability as an output. Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice because of its range.
- The function is differentiable and provides a smooth gradient, i.e., preventing jumps in output values. This is represented by an S-shape of the sigmoid activation function.

Limitations of Sigmoid- *Vanishing gradient* problem

- As shown in figure, the gradient values are only significant for range -3 to 3, and the graph gets much flatter in other regions.
- It implies that for values greater than 3 or less than -3, the function will have very small gradients. As the gradient value approaches zero, the network ceases to learn and suffers from the *Vanishing gradient problem*.

- The output of the logistic function is not symmetric around zero. So the output of all the neurons will be of the same sign. This makes the training of the neural network more difficult and unstable.

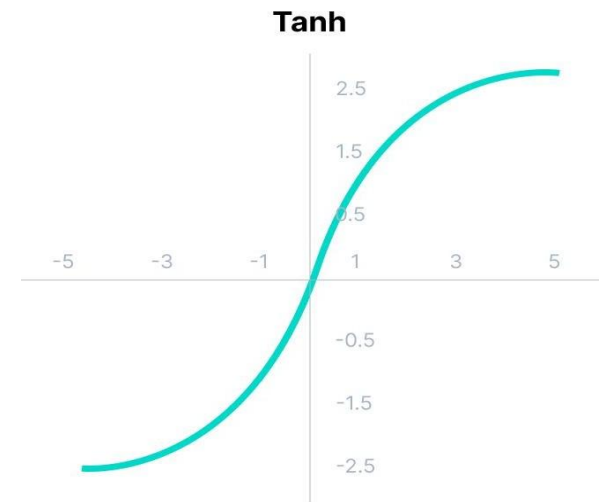


Tanh Function (Hyperbolic Tangent)

- Tanh function is very similar to the sigmoid/logistic activation function, and even has the same S-shape with the difference in output range of -1 to 1. In Tanh, the larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to -1.0.
- Advantages of using this activation function are:
 - The output of the tanh activation function is Zero centered; hence we can easily map the output values as strongly negative, neutral, or strongly positive.
 - Usually used in hidden layers of a neural network as its values lie between -1 to 1; therefore, the mean for the hidden layer comes out to be 0 or very close to it. It helps in centering the data and makes learning for the next layer much easier.

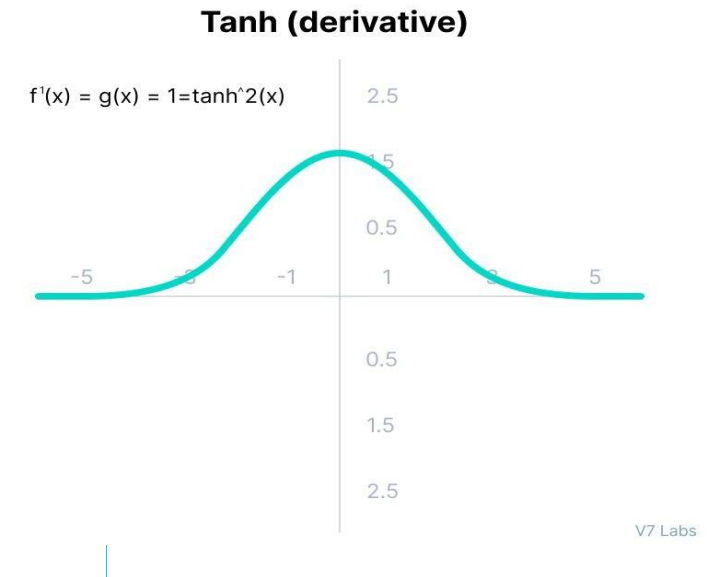
Tanh

$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$



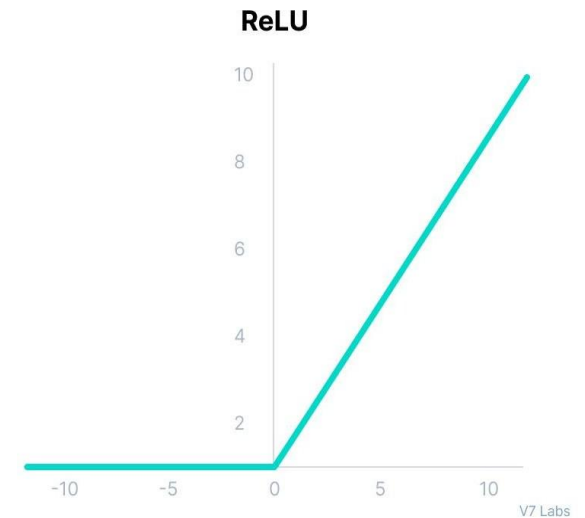
Gradient of the Tanh Activation Function

- Faces the problem of vanishing gradients similar to the sigmoid activation function. Plus the gradient of the tanh function is much steeper as compared to the sigmoid function.
- Although both sigmoid and tanh face vanishing gradient issue, tanh is zero centered, and the gradients are not restricted to move in a certain direction. Therefore, in practice, tanh nonlinearity is always preferred to sigmoid nonlinearity



ReLU Function-(Rectified Linear Unit Function)

- Although it gives an impression of a linear function, ReLU has a derivative function and allows for backpropagation while simultaneously making it computationally efficient.
- The main catch here is that the ReLU function does not activate all the neurons at the same time.
- The neurons will only be deactivated if the output of the linear transformation is less than 0.
- The advantages of using ReLU as an activation function are as follows:
 - Since only a certain number of neurons are activated, the ReLU function is far more computationally efficient when compared to the sigmoid and tanh functions.
 - ReLU accelerates the convergence of gradient descent towards the global minimum of the loss function due to its linear, non-saturating property.

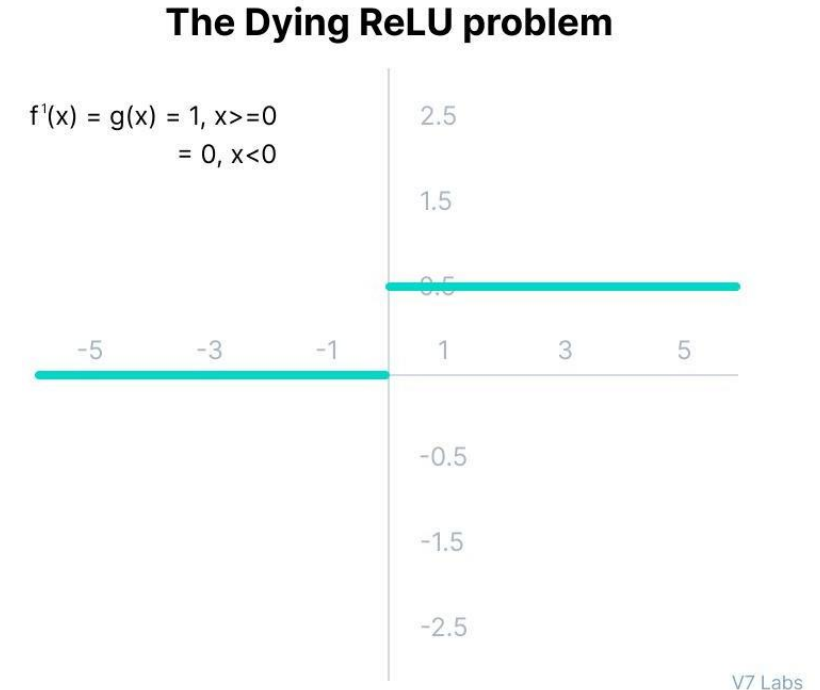


ReLU

$$f(x) = \max(0, x)$$

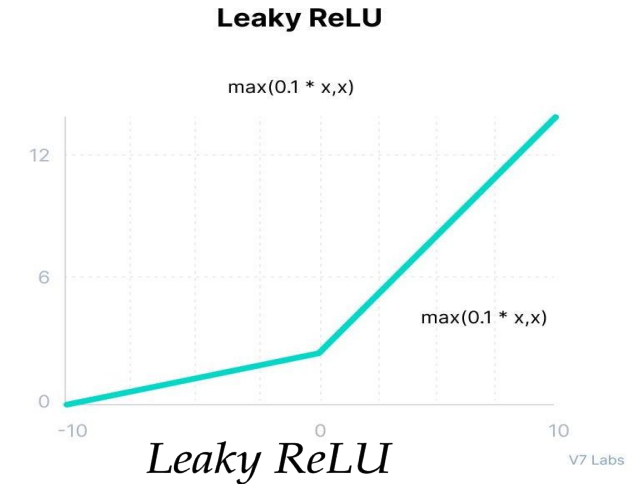
The Dying ReLU problem

- The negative side of the graph makes the gradient value zero. Due to this reason, during the backpropagation process, the weights and biases for some neurons are not updated. This can create dead neurons which never get activated.
 - All the negative input values become zero immediately, which decreases the model's ability to fit or [train from the data](#) properly.

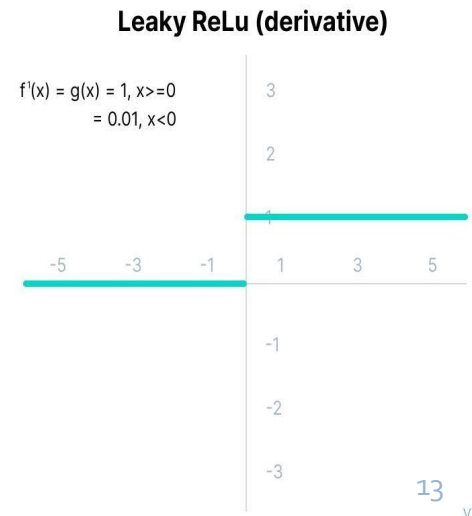


Leaky ReLU Function

- Leaky ReLU is an improved version of ReLU function to solve the Dying ReLU problem as it has a small positive slope in the negative area.
- The advantages of Leaky ReLU are same as that of ReLU, in addition to the fact that it does enable backpropagation, even for negative input values.
- By making this minor modification for negative input values, the gradient of the left side of the graph comes out to be a non-zero value. Therefore, we would no longer encounter dead neurons in that region.
- The limitations that this function faces include:
 - The predictions may not be consistent for negative input values
 - The gradient for negative values is a small value that makes the learning of model parameters time-consuming.

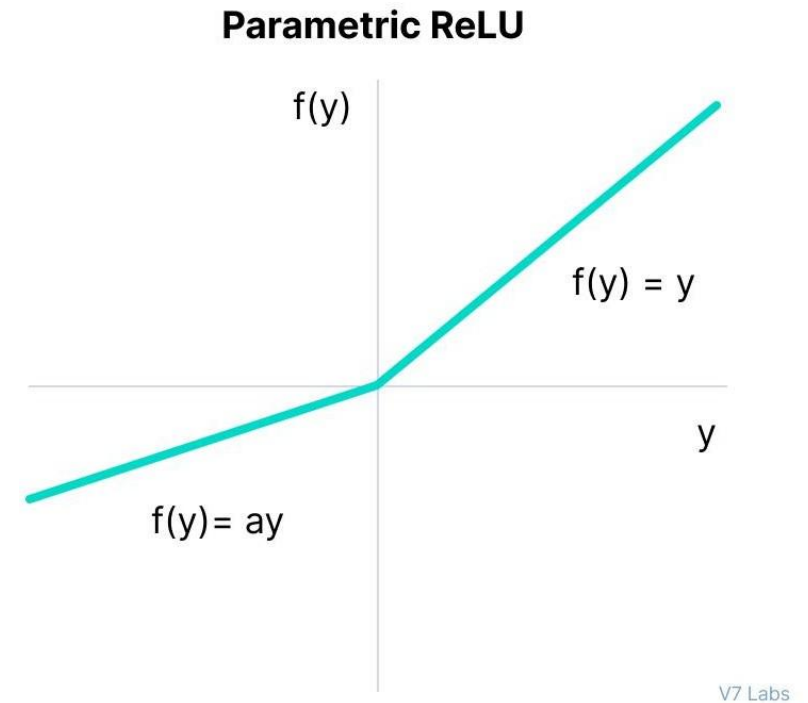


$$f(x) = \max(0.1x, x)$$



Parametric ReLU Function

- Parametric ReLU is another variant of ReLU that aims to solve the problem of gradient's becoming zero for the left half of the axis.
- This function provides the slope of the negative part of the function as an argument a . By performing backpropagation, the most appropriate value of a is learnt.
- Where " a " is the slope parameter for negative values.
- The parameterized ReLU function is used when the leaky ReLU function still fails at solving the problem of dead neurons, and the relevant information is not successfully passed to the next layer.
- This function's limitation is that it may perform differently for different problems depending upon the value of slope parameter a .



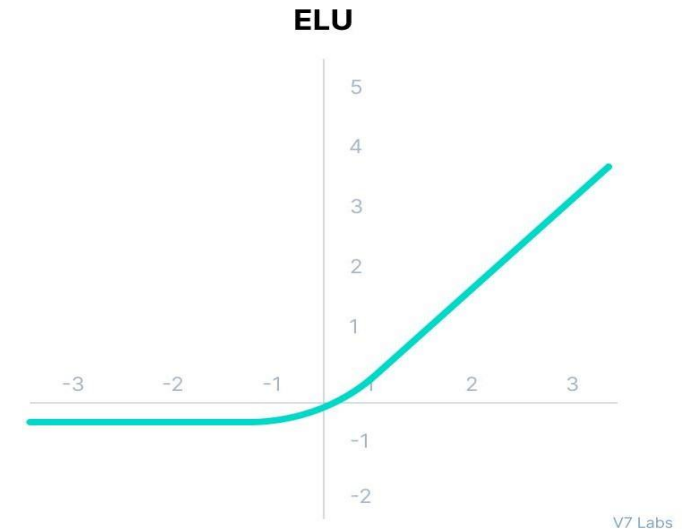
Parametric ReLU

$$f(x) = \max(ax, x)$$

Exponential Linear Units (ELUs)

Function

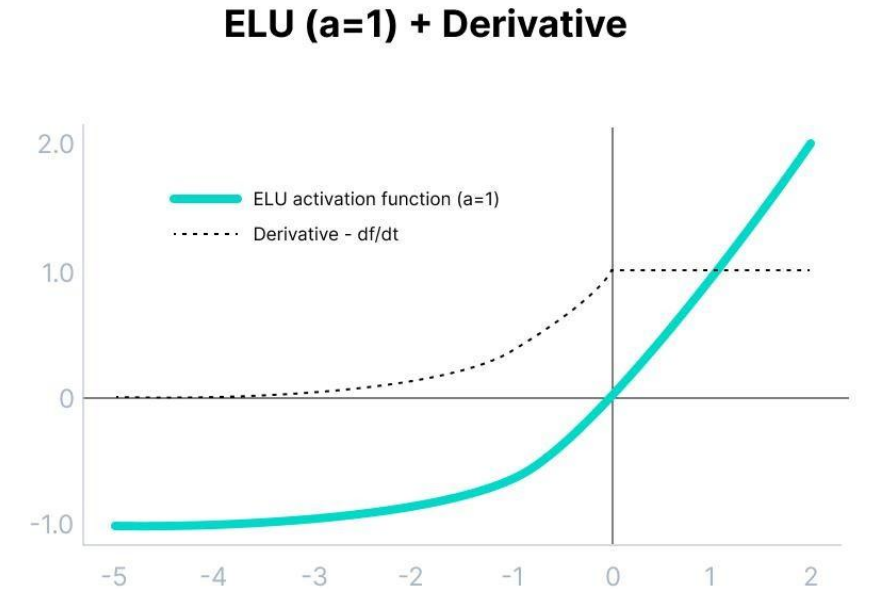
- Exponential Linear Unit, or ELU for short, is also a variant of ReLU that modifies the slope of the negative part of the function.
- ELU uses a log curve to define the negative values unlike the leaky ReLU and Parametric ReLU functions with a straight line.
- ELU is a strong alternative for f ReLU because of the following advantages:
 - ELU becomes smooth slowly until its output equal to $-\alpha$ whereas ReLU sharply smooths.
 - Avoids dead ReLU problem by introducing log curve for negative values of input. It helps the network nudge weights and biases in the right direction.



ELU

$$\begin{cases} x & \text{for } x \geq 0 \\ \alpha(e^x - 1) & \text{for } x < 0 \end{cases}$$

- The limitations of the ELU function are as follow:
 - It increases the computational time because of the exponential operation included
 - No learning of the 'a' value takes place
 - Exploding gradient problem



ELU derivative

V7 Labs

$$f'(x) = \begin{cases} 1 & \text{for } x \geq 0 \\ f(x) + \alpha & \text{for } x < 0 \end{cases}$$

Softmax Function

Probability

The output of the sigmoid function was in the range of 0 to 1, which can be thought of as probability.

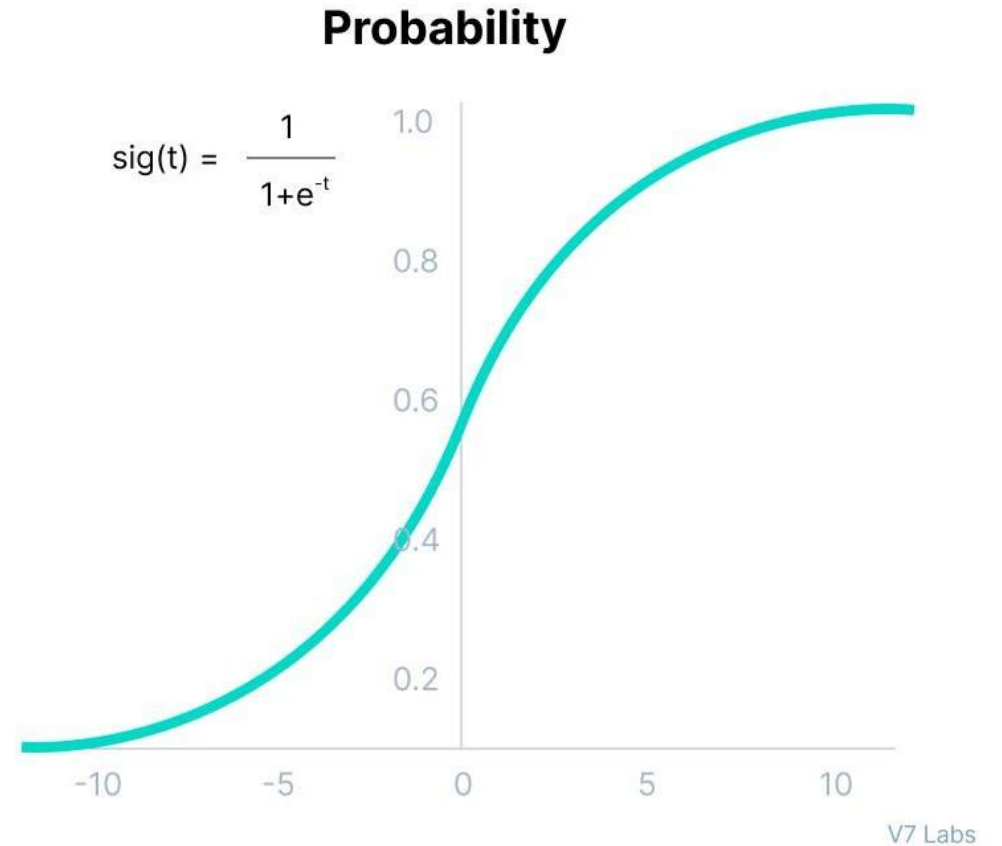
This function faces certain problems.

Let's suppose we have five output values of 0.8, 0.9, 0.7, 0.8, and 0.6, respectively.

How can we move forward with it?

The answer is: **We can't.**

The above values don't make sense as the sum of all the classes/output probabilities should be equal to 1.



Softmax Function

- Softmax function is described as a combination of multiple sigmoids.
- It calculates the relative probabilities.
- Similar to the sigmoid/logistic activation function, the SoftMax function returns the probability of each class.
- It is most commonly used as an activation function for the last layer of the neural network in the case of multi-class classification.
- Assume that you have three classes, meaning that there would be three neurons in the output layer. Now, suppose that your output from the neurons is [1.8, 0.9, 0.68].
- Applying the softmax function over these values to give a probabilistic view will result in the following outcome: [0.58, 0.23, 0.19].
- The function returns 1 for the largest probability index while it returns 0 for the other two array indexes. Here, giving full weight to index 0 and no weight to index 1 and index 2. So the output would be the class corresponding to the 1st neuron(index 0) out of three.
- Softmax activation function make things easy for multi-class classification problems.

Softmax

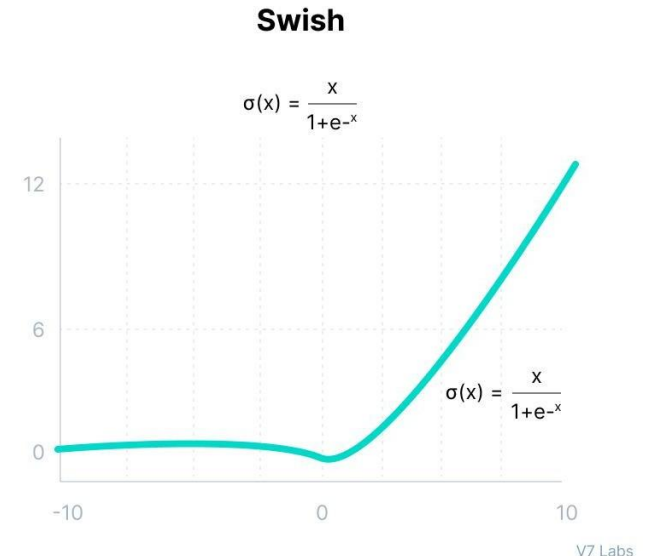
$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Swish

- It is a self-gated activation function developed by researchers at Google.
- Swish consistently matches or outperforms ReLU activation function on deep networks applied to various challenging domains such as [image classification](#), machine translation etc.

- **Swish Activation Function**

- This function is bounded below but unbounded above i.e. Y approaches to a constant value as X approaches negative infinity but Y approaches to infinity as X approaches infinity.



Swish

$$f(x) = x * \text{sigmoid}(x)$$

Why are deep neural networks hard to train?

•Vanishing Gradients

- Like the sigmoid function, certain activation functions squish an ample input space into a small output space between 0 and 1.
- Therefore, a large change in the input of the sigmoid function will cause a small change in the output. Hence, the derivative becomes small. For shallow networks with only a few layers that use these activations, this isn't a big problem.
- However, when more layers are used, it can cause the gradient to be too small for training to work effectively.

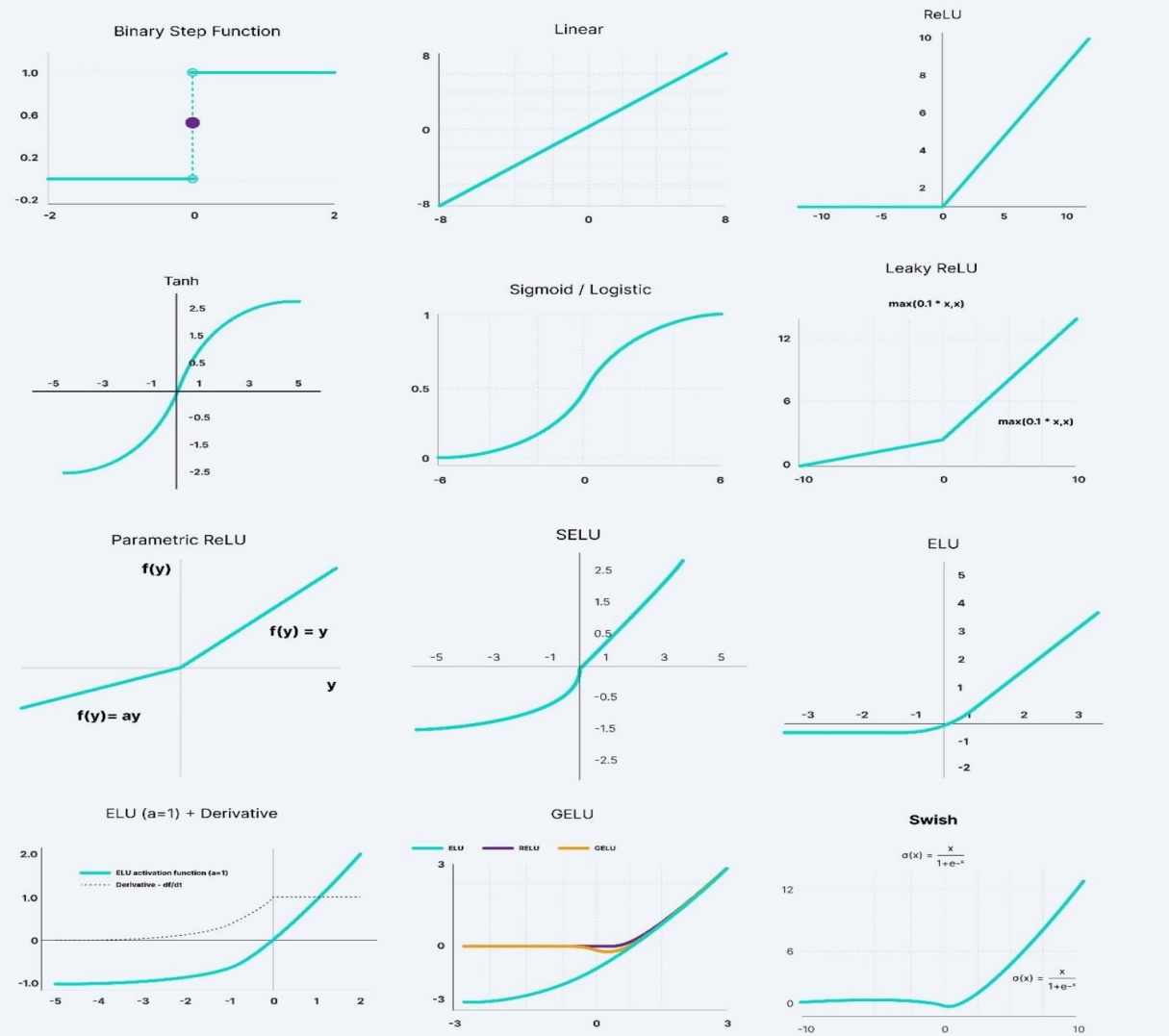
•Exploding Gradients

- Exploding gradients are problems where significant error gradients accumulate and result in very large updates to neural network model weights during training.
- An unstable network can result when there are exploding gradients, and the learning cannot be completed.
- The values of the weights can also become so large as to overflow and result in something called NaN values.

How to choose the right Activation Function?

1. ReLU activation function should only be used in the hidden layers.
 2. Sigmoid/Logistic and Tanh functions should not be used in hidden layers as they make the model more susceptible to problems during training (due to vanishing gradients).
 3. Swish function is used in neural networks having a depth greater than 40 layers.
- Finally, a few rules for choosing the activation function for your output layer based on the type of prediction problem that you are solving:
 1. **Regression** - Linear Activation Function
 2. **Binary Classification** - Sigmoid/Logistic Activation Function
 3. **Multiclass Classification** - Softmax
 - The activation function used in hidden layers is typically chosen based on the type of neural network architecture.
 5. **Convolutional Neural Network (CNN)**: ReLU activation function.
 6. **Recurrent Neural Network**: Tanh and/or Sigmoid activation function.

Neural Network Activation Functions



Neural Networks Activation Functions in a Nutshell

- Activation Functions are used to introduce non-linearity in the network.
- A neural network will almost always have the same activation function in all hidden layers. This activation function should be differentiable so that the parameters of the network are learned in backpropagation.
- ReLU is the most commonly used activation function for hidden layers.
- While selecting an activation function, you must consider the problems it might face: vanishing and exploding gradients.
- Regarding the output layer, we must always consider the expected value range of the predictions. If it can be any numeric value (as in case of the regression problem) you can use the linear activation function or ReLU.
- Use Softmax or Sigmoid function for the classification problems.

Loss Function

- The loss function, also referred to as the error function, is a crucial component in machine learning that quantifies the difference between the predicted outputs of a machine learning algorithm and the actual target values.
- For example, within a regression problem to predict car prices based on historical data, a loss function evaluates a neural network prediction based on a training sample from the training dataset. The loss function quantifies the gap or the error margin of the car price predicted by the network to the actual price.
- The resulting value, the loss, reflects the accuracy of the model's predictions. During training, a learning algorithm such as the backpropagation algorithm uses the gradients of the loss function with respect to the model's parameters to adjust these parameters and minimize the loss, effectively improving the model's performance on the dataset.
- The cost function, sometimes called the objective function, is an **average of the loss function of an entire training set** containing several training examples. The cost function quantifies the model's performance on the whole training dataset.

Loss Functions for Regression

Mean Square Error (MSE) / L2 Loss

- The Mean Square Error(MSE) or L2 loss is a loss function that quantifies the magnitude of the error between a machine learning algorithm prediction and an actual output by taking the average of the squared difference between the predictions and the target values. Squaring the difference between the predictions and actual target values results in a higher penalty assigned to more significant deviations from the target value. A mean of the errors normalizes the total errors against the number of samples in a dataset or observation.
- The mathematical equation for Mean Square Error (MSE) or L2 Loss is:

$$\text{MSE} = (1/n) * \sum (y_i - \bar{y})^2$$

Where:

- n is the number of samples in the dataset
- y_i is the predicted value for the i -th sample
- $\bar{y}$ is the target value for the i -th sample

Mean Absolute Error (MAE) / L1 Loss

- Mean Absolute Error (MAE), also known as L1 Loss, is a loss function used in regression tasks that calculates the average absolute differences between predicted values from a machine learning model and the actual target values. Unlike Mean Squared Error (MSE), MAE does not square the differences, treating all errors with equal weight regardless of their magnitude.
- The mathematical equation for Mean Absolute Error (MAE) or L1 Loss is:

$$\text{MAE} = (1/n) * \sum |y_i - \bar{y}|$$

Where:

- n is the number of samples in the dataset
- y_i is the predicted value for the i -th sample
- $\bar{y}$ is the target value for the i -th sample

Huber loss

- By definition, Huber loss is a robust loss function that combines the best properties of Mean Squared Error (MSE) and Mean Absolute Error (MAE), using in evaluating regression tasks.
- Mathematically, we can observe for residual with an absolute value less than or equal to the threshold parameter δ , the Huber loss is **quadratic**. For residuals with an absolute value greater than the threshold parameter δ , the Huber loss is **linear**.
- The hybrid nature of Huber loss makes it less sensitive to outliers, similar to MAE, but also penalizes small errors within the data points, similar to MSE. The following chart displays the visualization of the two loss functions.

$$L_{\delta}(a) = \begin{cases} \frac{1}{2}a^2 & \text{if } |a| \leq \delta \\ \delta(|a| - \frac{1}{2}\delta) & \text{if } |a| > \delta \end{cases}$$

where $a = y - f(x)$ is the residual, and δ is a threshold parameter.

Loss Functions for Classification

Binary Cross-Entropy Loss / Log Loss

- Binary Cross-Entropy Loss(BCE) is a performance measure for classification models that outputs a prediction with a probability value typically between 0 and 1, and this prediction value corresponds to the likelihood of a data sample belonging to a class or category. In the case of Binary Cross-Entropy Loss, there are two distinct classes. But notably, a variant of cross-entropy loss, Categorical Cross-Entropy applies to multiclass classification scenarios.
- To understand Binary Cross-Entropy Loss, sometimes called Log Loss, it is helpful to discuss the components of the terms.
- **Loss:** This is a mathematical quantification of the margin/difference between the prediction of a machine learning algorithm and the actual target value.
- **Entropy:** A simple definition of entropy is that it is a calculation of the degree of randomness or disorder within a system
- **Cross Entropy:** This is a term commonly utilised in information theory, and it measures the differences between two probability distributions that can be used to identify an observation.
- **Binary:** This is an expression of numerical digits using either of two states, 0 or 1. This is extended to the definition of Binary Classification where we distinguish two classes(A and B) using binary representation, where class A is assigned the numerical representation of 0 and class B is assigned 1.

$$L(y, f(x)) = -[y * \log(f(x)) + (1 - y) * \log(1 - f(x))]$$

Where:

- L represents the Binary Cross-Entropy Loss function
- y is the true binary label (0 or 1)
- f(x) is the predicted probability of the positive class (between 0 and 1)

Categorical cross-entropy

- Categorical cross-entropy is used when you have more than two classes in your classification problem (multi-class classification). It measures the difference between two probability distributions: the predicted probability distribution and the true distribution, which is represented by a one-hot encoded vector.
- In a one-hot encoded vector, the correct class is represented as "1" and all other classes as "0." Categorical cross-entropy penalizes predictions based on how confident the model is about the correct class.
- If the model assigns a high probability to the true class, the cross-entropy will be low. Conversely, if the model assigns low probability to the correct class, the cross-entropy will be high.

Hinge Loss

- Hinge Loss is a loss function utilized within machine learning to train classifiers that optimize to increase the margin between data points and the **decision boundary**. Hence, it is mainly used for **maximum margin** classifications. To ensure the maximum margin between the data points and boundaries, hinge loss penalizes predictions from the machine learning model that are wrongly classified, which are predictions that fall on the wrong side of the **margin boundary** and also predictions that are correctly classified but are within close proximity to the decision boundary.
- This characteristic of the Hinge Loss function ensures machine learning models are able to predict the accurate classification of data points to their target value with confidence that exceeds the threshold of the decision boundary. This approach to machine algorithm learning enhances the model's generalization capabilities, making it effective for accurately classifying data points with a high degree of certainty.
- **The mathematical equation for Hinge Loss is:**

$$L(y, f(x)) = \max(0, 1 - y * f(x))$$

Code

- <https://neptune.ai/blog/keras-loss-functions>
- https://colab.research.google.com/github/goodboychan/chans_jupyter/blob/main/_notebooks/2020-07-05-02-Loss-functions.ipynb

Thank You 🥰